

Understanding GridSearchCV and RandomizedSearchCV in scikit-learn

Model performance can often be improved by **tuning hyperparameters**—settings that control the behavior of the model. scikit-learn provides two powerful tools to do this systematically:

1. What Is Hyperparameter Tuning?

- **Hyperparameters** are values **set before training** (e.g., regularization strength `c` in `LogisticRegression`).
- **Tuning** means searching for the combination of hyperparameters that gives the best model performance.

2. GridSearchCV : Exhaustive Search

Tries **every possible combination** of hyperparameters you specify, using **cross-validation** to evaluate each one.

1. For each combination of parameters:
 - Fit the pipeline on $k-1$ folds
 - Evaluate on the remaining fold
 - Average performance across folds
2. Pick the combination with the best average score

Key Parameters:

```
GridSearchCV(  
    estimator,          # The model or pipeline  
    param_grid,         # Dictionary of hyperparameters to search  
    scoring=None,       # Metric (e.g., 'accuracy', 'f1', etc.)  
    cv=None,            # Cross-validation strategy (int or splitter object)  
    n_jobs=None,        # Number of parallel jobs (-1 = use all cores)  
    verbose=0,          # How much output to show  
    return_train_score=False,  
    refit=True          # Whether to retrain on full training set using best params  
)
```

3. RandomizedSearchCV : Random Sampling

Randomly samples a **fixed number** of parameter combinations from distributions you define.

When to Use:

- When the parameter space is **large**
- When you want **faster results**
- When some hyperparameters **don't matter much** and you want to save compute

Key Parameters:

```
RandomizedSearchCV(
    estimator,                # The model or pipeline
    param_distributions,       # Distributions to sample from (dict)
    n_iter=10,                # How many random combinations to try
    scoring=None,             # Evaluation metric
    cv=None,                  # Cross-validation strategy
    random_state=None,        # For reproducibility
    n_jobs=None,              # Parallel jobs
    verbose=0,
    return_train_score=False,
    refit=True
)
```

4. Cross-Validation (CV)

Both `GridSearchCV` and `RandomizedSearchCV` use **cross-validation** to estimate performance:

- `cv=5` : 5-fold cross-validation (default)
- You can pass custom strategies like `StratifiedKFold` , `GroupKFold` , etc.
- The goal is to get a **robust estimate** of model performance on unseen data

5. Choosing a Scoring Metric

You can define what “best” means:

- `'accuracy'` (default for classification)
- `'f1'` , `'recall'` , `'precision'` , `'roc_auc'` , `'neg_log_loss'`
- `'neg_mean_squared_error'` for regression
- You can also use `make_scorer` to create custom metrics

Example:

```
GridSearchCV(pipe, param_grid, cv=5, scoring='f1')
```

6. refit=True

After selecting the best hyperparameters via cross-validation, `refit=True` causes the model to be retrained **on the full training set** using those parameters.

```
best_model = grid.best_estimator_
```

This model is ready for evaluation on a **held-out test set**.

7. Accessing Results

After fitting:

Attribute	Description
<code>best_params_</code>	Best hyperparameter combination
<code>best_score_</code>	Best average CV score
<code>best_estimator_</code>	Full retrained model
<code>cv_results_</code>	Full performance of all tried combinations (as a dict of arrays)